

geracao

May 19, 2026

1 Geração de Números Aleatórios em Python

João Gabriel Miguêz da Silva - 2023008890

Felipe Bastos Vargas - 2023008881 ## 1.Introdução

Este trabalho tem como objetivo testar de diferentes modelos de distribuição para dados gerados aleatoriamente

Dentre as distribuições discretas serão aplicadas:

1. **Uniforme Discreta**
2. **Bernoulli**
3. **Binomial**
4. **Poisson**

Já as distribuições contínuas serão:

1. **Uniforme contínua**
2. **Normal**
3. **Exponencial**
4. **Gama**
5. **Beta**
6. **t-Student**

1.1 2.Metodologia

Inicialmente geramos aleatoriamente conjunto de dados usando a semente aleatória 42 para cada distribuição.

```
[84]: import numpy as np
      from scipy . stats import t
      import pandas as pd
      import math

      np.random.seed (42)

      # Distribuicoes discretas
      ddsUniformeDiscreta = np.random.randint(0,10,size=1000)      #uniforme
      ↪discreta
      ddsBernoulli = np.random.binomial(n =1,p =0.5,size =1000)      #Bernoulli
```

```

ddsBinomial = np.random.binomial(n =10,p =0.5,size =1000)           #Binomial
ddsPoisson = np.random.poisson(lam=5,size =1000)                   #Poisson

# Distribuicoes continuas
ddsUniformeContinua = np.random.uniform(0,1,size =1000)           #uniforme
↳ continua
ddsNormal = np.random.normal(0,1,size =1000)                      #normal
ddsExponencial = np.random.exponential(scale =2,size =1000)        #exponencial
ddsGamma = np.random.gamma(shape =2,scale =2,size =1000)          #gama
ddsBeta = np.random.beta( a =2,b =5,size =1000)                   #beta
ddsTStudent= t.rvs(df =5,size =1000)                              #
↳ t-Student

```

Em seguida analisamos as frequências observadas dos dados de cada conjunto. Para o conjunto de dados discretos, criamos um dicionário para que fosse possível contabilizar a frequência através de um loop.

1.1.1 2.1 Frequencias observadas dados discretos

```

[85]: #Frequencias dos dados Discretos
frqObservadaUniformeDiscreta = {}
frqObservadaBernoulli = {}
frqObservadaBinomial = {}
frqObservadaPoisson = {}

conjuntoDeDadosDiscretos = [(ddsUniformeDiscreta, frqObservadaUniformeDiscreta),
                             (ddsBernoulli, frqObservadaBernoulli),
                             (ddsBinomial, frqObservadaBinomial),
                             (ddsPoisson, frqObservadaPoisson)]

for dados, frq in conjuntoDeDadosDiscretos:
    for valor in dados:
        if valor in frq:
            frq[valor] += 1
        else:
            frq[valor] = 1

```

1.1.2 2.2 Frequencias observadas dados continuos

```

[86]: #Continuas
frqObservadaUniformeContinua = {}
frqObservadaNormal = {}
frqObservadaExponencial = {}
frqObservadaGamma = {}
frqObservadaBeta = {}

```

```

frqObservadaTStudent = {}

conjuntoDeDadosContinuos = [(ddsUniformeContinua, frqObservadaUniformeContinua),
                             (ddsNormal, frqObservadaNormal),
                             (ddsExponencial, frqObservadaExponencial),
                             (ddsGamma, frqObservadaGamma),
                             (ddsBeta, frqObservadaBeta),
                             (ddsTStudent, frqObservadaTStudent)]

```

1.1.3 2.3 Calculo dos Valores Esperados

2.3.1 Distribuições Discretas

2.3.1.1 Distribuição Uniforme Discreta

```

[87]: n = 10          # número de valores possíveis
      a = 0          # valor mínimo
      b = a + n - 1  # valor máximo possível (0 + 10 - 1 = 9)

def FMP_DiscretaUniforme(n, a, b):

    valores_x = [i for i in range(a, b + 1)]
    probabilidade = 1 / n
    probabilidades = [probabilidade for _ in range(n)]
    fmp_resultado = dict(zip(valores_x, probabilidades))

    return fmp_resultado

def calcular_estatisticas_uniforme(a, b):
    esperanca = (a + b) / 2
    variancia = ((b - a + 1)**2 - 1) / 12
    return esperanca, variancia

fmpDiscretaUniforme = FMP_DiscretaUniforme(n, a, b)
mediaTeoricaDiscUni, varTeoricaDiscUni = calcular_estatisticas_uniforme(a, b)

```

2.3.1.2 Distribuição de Bernoulli

```

[88]: pSucesso = 0.5
      pFracasso = 1 - pSucesso

def FMP_Bernoulli(p):

    valores_x = [0, 1]
    probabilidades = [1 - p, p]
    fmp_resultado = dict(zip(valores_x, probabilidades))
    return fmp_resultado

```

```
def calcular_estatisticas_bernoulli(p):
    esperanca = p
    variancia = p * (1 - p)
    return esperanca, variancia

fmpBernoulli = FMP_Bernoulli(pSucesso)
mediaTeoricaBernoulli, varTeoricaBernoulli =
    ↪calcular_estatisticas_bernoulli(pSucesso)
```

2.3.1.3 Distribuição Binomial

```
[89]: n = 10    # Número de tentativas independentes
      p = 0.5    # Probabilidade de sucesso em cada tentativa

def FMP_Binominal(n, p):
    fmp_resultado = {}
    for k in range(n + 1):
        coeficiente = math.comb(n, k)                #
        ↪coeficiente binomial:  $n! / (k! * (n - k)!)$ 
        probabilidade = coeficiente * (p**k) * ((1 - p)**(n - k))    # $P(X = k) =$ 
        ↪Coeficiente *  $p^k * (1-p)^{(n-k)}$ 
        # Guardar no dicionário
        fmp_resultado[k] = probabilidade
    return fmp_resultado

def calcular_estatisticas_binomial(n, p):
    esperanca = n * p
    variancia = n * p * (1 - p)
    return esperanca, variancia

fmpBinomial = FMP_Binominal(n, p)
mediaTeoricaBinomial, varTeoricaBinomial = calcular_estatisticas_binomial(n, p)
```

2.3.1.4 Distribuição Poisson

```
[90]: lam = 5    # Lambda

def FMP_Poisson(lam, limite_max=15):
    fmp_resultado = {}
    for k in range(limite_max + 1):

        fatorial_k = math.factorial(k)
        probabilidade = (math.exp(-lam) * (lam**k)) / fatorial_k    # $P(X = k) =$ 
        ↪ $(e^{-}) * \frac{\lambda^k}{k!}$ 

        # Guardar no dicionário
        fmp_resultado[k] = probabilidade
```

```

    return fmp_resultado

def calcular_estatisticas_poisson(lam):
    return lam, lam

fpmPoisson = FMP_Poisson(lam)
mediaTeoriaPoisson, varTeoricaPoisson = calcular_estatisticas_poisson(lam)

```

```

[91]: tabelas_frequencia = [
    ("Uniforme Discreta", frqObservadaUniformeDiscreta, fmpDiscretaUniforme),
    ("Bernoulli", frqObservadaBernoulli, fmpBernoulli),
    ("Binomial", frqObservadaBinomial, fmpBinomial),
    ("Poisson", frqObservadaPoisson, fpmPoisson)
]

for nome, frq_dict, fmp_dict in tabelas_frequencia:
    print(f"\n" + "="*70)
    print(f" TABELA COMPARATIVA: {nome.upper()} ")
    print("="*70)

    # 1. Cria o DataFrame a partir do dicionário de frequências observadas
    df = pd.DataFrame(list(frq_dict.items()), columns=['Valor', 'Freq. Obs. Absoluta (f_o)'])

    # 2. Ordena a tabela pelos valores (para ficar em ordem crescente: 0, 1, 2..)
    df = df.sort_values(by='Valor').reset_index(drop=True)

    # 3. Calcula o total de observações simuladas (deve dar 1000)
    total_observacoes = df['Freq. Obs. Absoluta (f_o)'].sum()

    # 4. Trazer a Probabilidade Teórica mapeando através do valor da chave do dicionário FMP
    df['Prob. Teórica P(X=x)'] = df['Valor'].map(fmp_dict)

    # Caso um valor observado não exista na FMP teórica (improvável, mas segurança), preenche com 0
    df['Prob. Teórica P(X=x)'] = df['Prob. Teórica P(X=x)'].fillna(0)

    # 5. Calcula a Frequência Esperada Absoluta (Probabilidade * Total de amostras)
    df['Freq. Esp. Absoluta (f_e)'] = df['Prob. Teórica P(X=x)'] * total_observacoes

    # 6. Calcula a Frequência Observada Relativa para comparar com a probabilidade teórica

```

```

df['Freq. Obs. Relativa (f_r)'] = df['Freq. Obs. Absoluta (f_o)'] /   

↳total_observacoes

# 7. Formatação visual (cópia limpa com porcentagens e arredondamentos)
df_formatado = df.copy()

# Deixando as frequências absolutas esperadas com 1 casa decimal (já que   

↳são teóricas)
df_formatado['Freq. Esp. Absoluta (f_e)'] = df_formatado['Freq. Esp.   

↳Absoluta (f_e)'].map("{:.1f}".format)

# Formatando probabilidades e frequências relativas como porcentagem
df_formatado['Prob. Teórica P(X=x)'] = df_formatado['Prob. Teórica P(X=x)']   

↳map("{:.2%}".format)
df_formatado['Freq. Obs. Relativa (f_r)'] = df_formatado['Freq. Obs.   

↳Relativa (f_r)'].map("{:.2%}".format)

# Organizando a ordem das colunas para facilitar a comparação visual direta
colunas_ordenadas = [
    'Valor',
    'Freq. Obs. Absoluta (f_o)', 'Freq. Esp. Absoluta (f_e)',
    'Freq. Obs. Relativa (f_r)', 'Prob. Teórica P(X=x)'
]

# Exibe a tabela final estruturada na tela
print(df_formatado[colunas_ordenadas].to_string(index=False))

# import pandas as pd

# tabelas_frequencia = [
#     ("Uniforme Discreta", frqUniformeDiscreta),
#     ("Bernoulli", frqBernoulli),
#     ("Binomial", frqBinomial),
#     ("Poisson", frqPoisson)
# ]

# for nome, frq_dict in tabelas_frequencia:
#     print(f"\n" + "="*40)
#     print(f" TABELA DE FREQUÊNCIA: {nome.upper()} ")
#     print("="*40)

# # 1. Cria o DataFrame a partir do dicionário de frequências
# df = pd.DataFrame(list(frq_dict.items()), columns=['Valor', 'Freq.   

↳Absoluta (f_i)'])

```

```

# # 2. Ordena a tabela pelos valores (para ficar em ordem crescente: 0, 1,
# ↪2...)
# df = df.sort_values(by='Valor').reset_index(drop=True)

# # 3. Calcula a Frequência Relativa (proporção/porcentagem)
# total_observacoes = df['Freq. Absoluta (f_i)'].sum()
# df['Freq. Relativa (f_r)'] = df['Freq. Absoluta (f_i)'] /
# ↪total_observacoes

# # 4. Calcula as Frequências Acumuladas
# df['Freq. Absoluta Acum. (F_i)'] = df['Freq. Absoluta (f_i)'].cumsum()
# df['Freq. Relativa Acum. (F_r)'] = df['Freq. Relativa (f_r)'].cumsum()

# # 5. Formata a exibição das colunas de proporção para porcentagem
# ↪(opcional, mas fica lindo)
# df_formatado = df.copy()
# df_formatado['Freq. Relativa (f_r)'] = df_formatado['Freq. Relativa
# ↪(f_r)'].map("{:.2%}".format)
# df_formatado['Freq. Relativa Acum. (F_r)'] = df_formatado['Freq. Relativa
# ↪Acum. (F_r)'].map("{:.2%}".format)

# # Exibe a tabela final na tela
# print(df_formatado.to_string(index=False))

```

=====

TABELA COMPARATIVA: UNIFORME DISCRETA

=====

Valor (f_r)	Freq. Obs. Teórica Prob. Teórica P(X=x)	Freq. Absoluta (f_o)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa
0		118		100.0
11.80%	10.00%			
1		83		100.0
8.30%	10.00%			
2		110		100.0
11.00%	10.00%			
3		94		100.0
9.40%	10.00%			
4		107		100.0
10.70%	10.00%			
5		96		100.0
9.60%	10.00%			
6		94		100.0
9.40%	10.00%			
7		100		100.0
10.00%	10.00%			

8		91	100.0
9.10%	10.00%		
9		107	100.0
10.70%	10.00%		

=====

TABELA COMPARATIVA: BERNOULLI

=====

Valor (f_r)	Freq. Obs. Prob. Teórica	Absoluta (f_o) P(X=x)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa
0		489	500.0	
48.90%	50.00%			
1		511	500.0	
51.10%	50.00%			

=====

TABELA COMPARATIVA: BINOMIAL

=====

Valor (f_r)	Freq. Obs. Prob. Teórica	Absoluta (f_o) P(X=x)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa
0		1	1.0	
0.10%	0.10%			
1		12	9.8	
1.20%	0.98%			
2		50	43.9	
5.00%	4.39%			
3		125	117.2	
12.50%	11.72%			
4		175	205.1	
17.50%	20.51%			
5		246	246.1	
24.60%	24.61%			
6		218	205.1	
21.80%	20.51%			
7		115	117.2	
11.50%	11.72%			
8		44	43.9	
4.40%	4.39%			
9		12	9.8	
1.20%	0.98%			
10		2	1.0	
0.20%	0.10%			

=====

TABELA COMPARATIVA: POISSON

=====

Valor (f_r)	Freq. Obs. Prob. Teórica	Absoluta (f_o) P(X=x)	Freq. Esp. Absoluta (f_e)	Freq. Obs. Relativa
----------------	-----------------------------	--------------------------	------------------------------	---------------------

0		7	6.7
0.70%	0.67%		
1		31	33.7
3.10%	3.37%		
2		88	84.2
8.80%	8.42%		
3		147	140.4
14.70%	14.04%		
4		177	175.5
17.70%	17.55%		
5		173	175.5
17.30%	17.55%		
6		151	146.2
15.10%	14.62%		
7		84	104.4
8.40%	10.44%		
8		67	65.3
6.70%	6.53%		
9		45	36.3
4.50%	3.63%		
10		15	18.1
1.50%	1.81%		
11		8	8.2
0.80%	0.82%		
12		5	3.4
0.50%	0.34%		
13		1	1.3
0.10%	0.13%		
15		1	0.2
0.10%	0.02%		